

Geração de Código

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

28 de maio de 2026

Observações em Relação a um Compilador de Produção

- A fase de geração de código recebe como entrada a representação intermediária (RI) do programa-fonte juntamente com a tabela de símbolos e produz um código-objeto semanticamente equivalente ao programa fonte.
- Para gerar código objeto eficiente, os compiladores incluem uma fase de otimização antes da geração de código. Nesta fase, pode ser necessário converter de uma RI em outra, e realizar diversas otimizações ainda a nível de RI.
- um gerador de código é composto por três tarefas principais: seleção de instruções, alocação e atribuição de registradores e escalonamento de instrução.
- A fase de geração de código é complexa. É muito dependente da RI utilizada e da arquitetura da máquina alvo.

Observações em Relação a um Projeto Didático

- A disciplina tem duração de um semestre (na prática, quatro meses)
- O objetivo é gerar um compilador de caráter didático que possibilite mostrar ao aluno as fases de um compilador completo, sem incluir a otimização de código: análise léxico-sintática, análise semântica e geração da RI, tradução da RI em uma linguagem assembly e execução do código gerado em um simulador.
- Portanto, devido ao caráter puramente didático e ao prazo para se obter um código objeto, várias simplificações foram feitas e praticamente omitimos as principais técnicas de geração de código necessárias em um compilador de produção (ver Capítulo 8 do Dragon Book para uma visão detalhada desta fase.)

Simplificações Consideradas

- Utilizamos a árvore sintática gerada na fase de análise como RI.
- Consideramos que o código objeto será executado em uma máquina de pilha, conforme será explicado a seguir.
- Utilizaremos uma arquitetura MIPS para simular a máquina de pilha. Isso permite uma grande simplificação na geração de código para um projeto didático, mas é uma abordagem totalmente sem sentido para um compilador de produção, pois estamos forçando uma máquina baseada em registradores a usar excessivamente a memória.
- Escolhemos uma arquitetura MIPS devido à simplicidade de suas instruções. Os alunos precisam aprender apenas um sub-conjunto pequenos do conjunto reduzido de instruções do MIPS.

Stack machine

- Uma máquina de Pilha é um modelo de máquina cuja única memória disponível funciona como uma pilha.
- De modo genérico cada instrução de máquina pode ser vista como uma expressão do tipo $r = f(a_1, a_2, \dots a_n)$.
- r é o resultado da instrução, f representa uma operação e a_i são os argumentos da operação f .
- Em uma máquina de pilha, os n operandos se encontram na pilha.
- Para executar a instrução a máquina desempilha pares de parâmetros no topo da pilha, aplica a operação f sobre os parâmetros desempilhados e empilha o resultado, até que o resultado final r fique no topo da pilha.

- Suponha a operação de soma de dois números: $3 + 4$.
- Supondo que no topo da pilha estejam os dois operandos: $4, 3, X$, onde o elemento do topo é 4 e X representa o restante dos elementos na pilha.
- A soma remove os dois elementos do topo, deixando a pilha com a seguinte configuração: X .
- Os valores desempilhados são somados e o resultado é empilhado: $7, X$.
- importante notar a seguinte propriedade da máquina de pilha: o dado X abaixo dos parâmetros, que existia antes da operação, permanece após a operação.

Característica

- As instruções da máquina de pilha não contêm endereços explícitos, pois os operandos sempre se encontram nas últimas posições no topo da pilha.
- Em contraste, uma máquina com registradores, necessita indicar onde estão os operandos:
 - add r1,r2,r3
- Programas em máquinas de pilha tendem a ser mais compactos, pois o número de instruções é menor. Isto é uma das razões porque o *Bytecode* corresponde a uma máquina de pilha.

Característica

- As máquinas de registradores por outro lado são bem mais rápidas, pois o acesso aos registradores é muito mais rápido do que o acesso à memória.
- Uma máquina intermediária é uma máquina de pilha com n registradores, a qual se aproxima mais das arquiteturas atuais.
- Um caso particular é uma máquina de pilha com um registrador (acumulador).

Acumulador

- Em uma máquina de pilha pura, a instrução de soma implica em três acessos à memória: 2 pop e 1 push.
- Em uma máquina de pilha com o acumulador, um operando fica no acumulador e outro fica no topo da pilha.
- Para somar dois números, o valor no topo da pilha é desempilhado e somado com o valor do acumulador.
- O resultado fica armazenando no acumulador.

Acumulador

- Considere a expressão $op(e_1, e_2, \dots, e_n)$, onde cada $e_i, 1 \leq i \leq n$ é uma expressão na linguagem de alto nível.
- Para cada $e_i, (1 \leq i < n)$ compute e_i . O resultado de e_i fica no acumulador. Empilhe o valor do acumulador.
- Para a última expressão e_n compute a expressão e deixe o resultado no acumulador (o valor não é empilhado)
- Em seguida repita as operações seguintes $n - 1$ vezes:
 - Desempilhe o elemento do topo e aplique o operador op sobre esse elemento e o valor que está no acumulador.
 - O resultado fica no acumulador.
- **Invariante:** Após avaliar $op(e_1, e_2, \dots, e_n)$, a pilha permanece inalterada, contendo o mesmo dado que continha antes da avaliação da expressão - a avaliação da expressão preserva a pilha.

Exemplo

$3 + (2 + 4)$

Código

Acumulador

Pilha

Exemplo

$3 + (2 + 4)$

Código

$acc \leftarrow 3$

Acumulador

3

Pilha

?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?
push acc	2	2, 3, ?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?
push acc	2	2, 3, ?
$acc \leftarrow 4$	4	2, 3, ?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?
push acc	2	2, 3, ?
$acc \leftarrow 4$	4	2, 3, ?
$acc \leftarrow acc + top$	6	2, 3, ?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?
push acc	2	2, 3, ?
$acc \leftarrow 4$	4	2, 3, ?
$acc \leftarrow acc + top$	6	2, 3, ?
pop	6	3, ?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?
push acc	2	2, 3, ?
$acc \leftarrow 4$	4	2, 3, ?
$acc \leftarrow acc + top$	6	2, 3, ?
pop	6	3, ?
$acc \leftarrow acc + top$	9	3, ?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?
push acc	2	2, 3, ?
$acc \leftarrow 4$	4	2, 3, ?
$acc \leftarrow acc + top$	6	2, 3, ?
pop	6	3, ?
$acc \leftarrow acc + top$	9	3, ?
pop	9	?

Exemplo

$3 + (2 + 4)$

Código	Acumulador	Pilha
$acc \leftarrow 3$	3	?
push acc	3	3, ?
$acc \leftarrow 2$	2	3, ?
push acc	2	2, 3, ?
$acc \leftarrow 4$	4	2, 3, ?
$acc \leftarrow acc + top$	6	2, 3, ?
pop	6	3, ?
$acc \leftarrow acc + top$	9	3, ?
pop	9	?

Gerando código para expressões em máquinas de pilha

- Geraremos código para uma máquina (virtual) de pilha com um registrador.
- O código resultante deve executar em uma máquina real (MIPS) ou em um simulador de uma máquina real.
- Então, simularemos as instruções de uma máquina de pilha com um registrador, usando instruções e registradores do MIPS.

MIPS

- MIPS - Reduced Instruction Set - arquitetura antiga (década de 80).
- Conjunto reduzido de instruções.
- A maioria das instruções são realizadas com registradores.
- Usa instruções load e store para transferir dados entre registradores e memória.
- Possui 32 registradores de propósito geral.
 - Utilizaremos: \$a0, \$fp, \$s0, \$s1, \$sp, \$t1, \$v0
 - Ler bibliografia indicada.

- Utilizaremos o registrador \$s0 do MIPS como o acumulador.
- A Pilha será mantida em memória.
 - A pilha cresce em direção aos endereços mais baixos.
 - Isso é uma convenção padrão na programação do MIPS.
- O endereço da próxima localização de memória é mantido no registrador \$sp.
- \$sp aponta para a posição seguinte ao topo da pilha. Portanto, o topo da pilha está na posição $\$sp+4$.

Instruções inicialmente utilizadas

- `lw  $reg_1$ , offset( $reg_2$ )`
 - Carrega em reg_1 a palavra de 32 bits que está na posição $reg_2 + offset$ de memória.
- `add  $reg_1$ ,  $reg_2$ ,  $reg_3$`
 - $reg_1 \leftarrow reg_2 + reg_3$
- `sw  $reg_1$ , offset( $reg_2$ )`
 - Armazena a palavra que está no registrador reg_1 no endereço de memória indicado por $reg_2 + offset$
- `addiu  $reg_1$ ,  $reg_2$ ,  $imm$`
 - $reg_1 \leftarrow reg_2 + imm$ (u - underflow não é verificado)
- `li  $reg_1$ ,  $imm$`
 - $reg_1 \leftarrow imm$

Exemplo

- Código de máquina de pilha para a expressão $7 + 5$ usando MIPS

- $acc \leftarrow 7$
- push acc
- $acc \leftarrow 5$
- $acc \leftarrow acc + top$
- pop
- li \$s0, 7
- sw \$s0, 0(\$sp)
- addiu \$sp, \$sp, -4
- li \$s0, 5
- lw \$t1, 4(\$sp)
- add \$s0, \$s0, \$t1
- addiu \$sp, \$sp, 4

Sistematizando a geração de código

- Apresentamos a seguir algumas técnicas para sistematizar a geração de código para construções de linguagens de alto nível.
- Utilizaremos a máquina de pilha implementada em MIPS para facilitar a geração de código.
- As regras de adotadas para programação em máquina de pilha que vimos anteriormente possibilitam sistematizar e até mesmo automatizar a geração de código, como veremos a seguir.
- Iniciamos com a geração de código para expressões da linguagem de alto nível.

- Para cada expressão e geramos código MIPS que:
 - Calcula o valor de e e deixa o valor armazenado em $\$s0$ (nosso acumulador).
 - Preserva $\$sp$ e o conteúdo da pilha.
 - definimos uma função de alto nível $cgenEx(e)$ que gera o código que implementa a expressão e .
- A função $cgenEx(e)$ deve tratar dois tipos distintos de expressões.

cgenEx(*e*)

- Caso 1: a expressão é uma constante *i*:
 - *cgenEx*(*i*) = `print ("li $s0 i")`
 - Preserva a pilha !

$cgenEx(e)$

- Caso 2: $cgenEx(e_1 + e_2)$:
- $cgenEx(e_1 + e_2) = \{$
 - $cgenEx(e_1)$ - chamamos $cgenEx$ para gerar código para e_1 .
Ao ser executado esse código deve deixar o resultado de e_1 em $\$s0$.

cgenEx(*e*)

- Caso 2: *cgenEx*($e_1 + e_2$):
- *cgenEx*($e_1 + e_2$) = {
 - *cgenEx*(e_1) - chamamos *cgenEx* para gerar código para e_1 . Ao ser executado esse código deve deixar o resultado de e_1 em \$s0.
 - `print ("sw $s0, 0($sp)"); print ("addiu $sp, $sp -4");` - empilhamos o conteúdo de \$s0 (e_1) antes de gerar código para e_2 .

cgenEx(*e*)

- Caso 2: *cgenEx*($e_1 + e_2$):
- *cgenEx*($e_1 + e_2$) = {
 - *cgenEx*(e_1) - chamamos *cgenEx* para gerar código para e_1 . Ao ser executado esse código deve deixar o resultado de e_1 em \$s0.
 - `print ("sw $s0, 0($sp)"); print ("addiu $sp, $sp -4");` - empilhamos o conteúdo de \$s0 (e_1) antes de gerar código para e_2 .
 - *cgenEx*(e_2) - chamamos *cgenEx* para gerar código para e_2 . O código gerado deve colocar o resultado de e_2 em \$s0.

cgenEx(*e*)

- Caso 2: *cgenEx*($e_1 + e_2$):
- *cgenEx*($e_1 + e_2$) = {
 - *cgenEx*(e_1) - chamamos *cgenEx* para gerar código para e_1 . Ao ser executado esse código deve deixar o resultado de e_1 em \$s0.
 - `print ("sw $s0, 0($sp)"); print ("addiu $sp, $sp -4");` - empilhamos o conteúdo de \$s0 (e_1) antes de gerar código para e_2 .
 - *cgenEx*(e_2) - chamamos *cgenEx* para gerar código para e_2 . O código gerado deve colocar o resultado de e_2 em \$s0.
 - `print("lw $t1, 4($sp)");` - copia o valor de e_1 que está no topo da pilha em \$t1.

cgenEx(*e*)

- Caso 2: *cgenEx*($e_1 + e_2$):
- *cgenEx*($e_1 + e_2$) = {
 - *cgenEx*(e_1) - chamamos *cgenEx* para gerar código para e_1 . Ao ser executado esse código deve deixar o resultado de e_1 em \$s0.
 - `print ("sw $s0, 0($sp)"); print ("addiu $sp, $sp -4");` - empilhamos o conteúdo de \$s0 (e_1) antes de gerar código para e_2 .
 - *cgenEx*(e_2) - chamamos *cgenEx* para gerar código para e_2 . O código gerado deve colocar o resultado de e_2 em \$s0.
 - `print("lw $t1, 4($sp)");` - copia o valor de e_1 que está no topo da pilha em \$t1.
 - `print("addiu $sp, $sp, 4");` - pop - preserva a pilha.

cgenEx(*e*)

- Caso 2: *cgenEx*($e_1 + e_2$):
- *cgenEx*($e_1 + e_2$) = {
 - *cgenEx*(e_1) - chamamos *cgenEx* para gerar código para e_1 . Ao ser executado esse código deve deixar o resultado de e_1 em \$s0.
 - `print ("sw $s0, 0($sp)"); print ("addiu $sp, $sp -4");` - empilhamos o conteúdo de \$s0 (e_1) antes de gerar código para e_2 .
 - *cgenEx*(e_2) - chamamos *cgenEx* para gerar código para e_2 . O código gerado deve colocar o resultado de e_2 em \$s0.
 - `print("lw $t1, 4($sp)");` - copia o valor de e_1 que está no topo da pilha em \$t1.
 - `print("addiu $sp, $sp, 4");` - pop - preserva a pilha.
 - `print("add $s0, $s0, $t1");` - soma o valor de e_1 com o valor de e_2 que está em \$s0 e deixa o resultado em \$s0.

Formato geral

- O algoritmo para $cgenEx(e)$ dado anteriormente pode servir de formato para geração de outras expressões aritméticas como subtração, multiplicação e divisão.
- O que é necessário é substituir o comando `print("add $0, $t1, $s0")` por um conjunto de comandos que gere código para a operação correspondente.
- Pode ser utilizado em caminhamento na árvore abstrata gerada como representação intermediária.

- *Branch on Equal* - desvia o controle para "label" se $reg_1 = reg_2$: `beq reg1, reg2, label`
- *Branch on Not Equal* - desvia o controle para "label" se $reg_1 \neq reg_2$: `bne reg1, reg2, label`
- *Branch on Less Than* - desvia o controle para "label" se $reg_1 < reg_2$: `blt reg1, reg2, label`
- *Branch on Greater Than* - desvia o controle para "label" se $reg_1 > reg_2$: `bgt reg1, reg2, label`
- *Branch on Less Than or Equal* - desvia o controle para "label" se $reg_1 \leq reg_2$: `ble reg1, reg2, label`
- *Branch on Greater Than or Equal* - desvia o controle para "label" se $reg_1 \geq reg_2$: `bge reg1, reg2, label`

Instruções de Desvio Incondicional

- *Branch* - vá para o "label":

b label

- *Jump* - vá para o "label":

j label

Geração de Código para IF-THEN-ELSE

cgenIf($e_1, e_2, c_{then}, c_{else}$) {

- Gera código para e_1 : *cgenEx*(e_1);
- Imprime instruções para empilhar e_1 : `print("sw $s0, 0($sp)");`
`print("addiu $sp, $sp, -4");`
- Gera código para e_2 : *cgenEx*(e_2);
- Imprime instruções para desempilhar o valor de e_1 em $\$t1$: `print ("lw $t1,`
`4($sp)");` `print("addiu $sp, $sp, 4");`
- Imprime comparação de e_1 com e_2 e desvio para o código do then:
`print("beq $s0, $t1, then");` `print ("else:");`
- Gera código para os comandos após *else*: *cgenCmd*(c_{else})
- Imprime desvio para o fim do if: `print("b End_if");`
- Imprime o rótulo para o início dos comandos correspondentes a *then*:
`print("then:");`
- Gera código para os comando do *then*: *cgenCmd*(c_{then});
- Imprime rótulo para o fim do if: `print("End_if:");`

}

Geração de Código para WHILE

cgenWhile(e_1, e_2, c_{while}) {

- Imprime rotulo para o inicio do while: `print ("while:");`
- Gera código para e_1 : `cgenEx( $e_1$ );`
- Imprime instruções para empilhar e_1 : `print("sw $s0, 0($sp)");`
`print("addiu $sp, $sp, -4");`
- Gera código para e_2 : `cgenEx( $e_2$ );`
- Imprime instruções para desempilhar o valor de e_1 em t_1 : `print ("lw $t1,`
`4($sp)");` `print("addiu $sp, $sp, 4");`
- Imprime comparação de e_1 com e_2 e desvio para o fim do comando
while: `print("bne $s0, $t1, fim_while");`
- Gera código para os comandos dentro do while: `cgenCmd( $c_{while}$ )`
- Imprime desvio para o início do while: `print("b while");`
- Imprime o rótulo para fim o do comando while : `print("fim_while:");`

}

Instruções e registradores adicionais

- A instrução `jal label` (*jump and link*) é utilizada para fazer chamada à função cujo código tem início a partir do endereço com rótulo *label*.
 - A instrução `jal` salva no registrador `$ra` - *return address* o endereço da instrução que a segue no código (i. e. o endereço de retorno - o *link* com o código chamador). Em seguida, realiza um desvio para *label*.
- A instrução `jr $ra` é executada ao final da função chamada para retornar o controle da execução para instrução imediatamente após a instrução `jal` executada na chamada.
- O registrador `$fp` (*frame pointer*) é utilizado para marcar o início de um frame. Seu uso é opcional na geração de código para funções.

Códigos de controle para a execução de uma função

- Para que uma chamada de função possa ocorrer, é necessário que o compilador gere códigos para empilhar os elementos do *frame* da função chamada.
- As instruções que formam esse código são denominadas *sequência de chamada*.
- Parte da sequência de chamada fica no código objeto da função ou programa que chama a função a ser executada.
- Essa parte da sequência de chamada é denominada *sequência de chamada do código chamador*.
- A outra parte da sequência de chamada fica no código objeto da função chamada e é denominada *sequência de chamada do código chamado*.

Sugestão de sequência de chamada no código chamador

- Seja uma chamada a uma função f passando n expressões como argumentos de entrada: $f(e_1, e_2, \dots, e_n)$.
- Sugerimos a seguinte sequência para implementar a chamada da função f :
- O código chamador:
 - Empilha o valor atual do frame pointer (\$fp). Esse valor corresponde ao ponteiro para o frame do próprio código chamador.
 - gera código para cada expressão correspondente a cada um dos n parâmetros e empilha o valor resultante. Expressões são geradas na ordem inversa.

Sequência de chamada do código chamador no MIPS

- $cgenf(f(e_1, e_2, \dots, e_n))$

```
print("sw $fp, 0($sp)") # empilha o valor de $fp
print("addiu $sp, $sp, -4")
#gera código para as expressões na ordem inversa e empilha
os resultados
cgen( $e_n$ )
print("sw $s0, 0($sp)")
print("addiu $sp, $sp, -4")
:
cgen( $e_1$ )
print("sw $s0, 0($sp)")
print("addiu $sp, $sp, -4")
#gera chamada à função
print("jal f_entry")
```

Sequência de chamada de função

Estrutura parcial do registro de ativação

A figura abaixo mostra como fica o registro de ativação após a execução da sequência de instruções efetuadas pelo código chamador

	\$fp do chamador
	valor de e_n
	valor de e_{n-1}
	$\vdots$
	valor de e_1
\$sp →	

Sequência de chamada de função

Sugestão de sequência de chamada no código chamado.

- Convencionaremos que o valor retornado por uma função sempre fica no acumulador (\$s0).
- O código chamado:
 - Empilha o valor de \$ra.
 - Atualiza o \$fp para apontar para o topo da pilha (para o valor de \$ra empilhado).
 - Aloca espaço na pilha para suas variáveis locais (se existirem).
 - Gera código para seus comandos.

Sequência de chamada de função

Sequência de chamada do código chamado em MIPS

`print("move $fp, $sp")` - `$fp` passa a apontar a posição após a última expressão - `move` é uma pseudo instrução do montador do MIPS que permite copiar o valor de um registrador (`$sp`) para outro registrador (`$fp`)

`print("sw $ra, 0($sp)")` # empilha `$ra`

`print("addiu $sp, $sp, -4")`

aloca espaço na pilha para as m variáveis locais de f , $m \geq 0$:

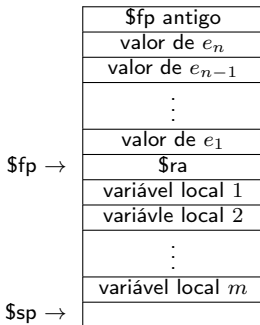
`print("addiu $sp, $sp,",  $-4 \times m$ )` # `$sp` aponta para a próxima posição após a m -ésima variável.

gera código para os comandos de f

Sequência de chamada de função

Estrutura do registro de ativação

A figura abaixo mostra como fica o registro de ativação após a execução da sequência de instruções efetuadas pelo código chamado



Características

- A forma como foi projetado, o RA:
 - Permite fácil acesso aos argumentos de chamada a partir do \$fp
 - Permite fácil acesso às variáveis locais, também a través do \$fp
 - Permite fácil acesso ao valor de \$ra a ser restaurado, também através do \$fp.
 - Mas, no nosso caso, o \$fp não aponta para o início do RA, mas para a região após os valores dos parâmetros da função.

Código para retorno de procedimento

Sequência no código chamado

- O valor de \$ra no registro de ativação deve ser restaurado.
- O valor do frame pointer que existia no momento chamada deve ser restaurado.
- O stack pointer deve voltar a apontar para o endereço que apontava antes da chamada.

```
print("lw $ra, 0($fp)"# restaura o valor do $ra
print("move $sp, $fp" # Desempilha variáveis locais.
print("addiu $sp, $sp", z) # onde  $z = (n + 1) \times 4$  (  $n$  é o
núm. de parâmetros)
print("lw $fp, 0($sp)" #restaura o valor de $fp do
chamador
print("jr $ra" ) #executa o retorno
```

Variáveis globais ao programa como um todo

- Pode-se utilizar um registrador (\$s1) para armazenar o início das variáveis globais.
- O valor de \$sp no início da geração de código é copiado em \$s1
- O valor de \$s1 não é alterado durante a execução do programa.
- O \$sp é deslocado para baixo na pilha para cada variável global.
- O deslocamento é igual ao tamanho da variável.
- Na tabela de símbolos, são armazenados para cada variável global: o seu escopo (0) e sua posição em relação ao início da lista de variáveis.
- Uma variável array de tamanho n conta como n variáveis escalares em termos de posicionamento na lista.

Acesso a variáveis globais

- O acesso a uma variável global g é feito do seguinte modo:
 - Pesquisar pelo nome de g na tabela de símbolos. A entrada para g na tabela contém o escopo da variável (0) e sua posição na lista de variáveis globais.
 - Seja p a posição de g na lista; o acesso a g é feito a partir de $\$s1$ do seguinte modo: $-(p - 1) * 4(\$s1)$.
 - O novo campo escopo na tabela de símbolos define o ponto de referência a para o deslocamento até a variável na memória: $escopo = 0$ - a referência é $\$s1$; $escopo > 0$ - a referência é o $\$fp$.

Por exemplo:

- Armazenamento em g :
`print ("sw $s0,",  $-(p - 1) * 4$ , "($s1)")`
- Carga a partir de g :
`print("lw $s0,",  $-(p - 1) * 4$ , "($s1)")`

Exemplo

- Suponha as seguintes declarações globais: `x: int; y[4]:int; z:int;`
- Na tabela de símbolos, os valores de posição para cada variável são: 1, 2, 6 (z inicia na posição 6, dado que y ocupa 4 posições).
- O campo escopo de cada entrada na tabela de símbolos é zero, logo o deslocamento é referente ao endereço em \$s1.
- A geração de código para armazenamento em z:
`print("sw $s0, ", -20, "($s1)")`
- A geração de código para armazenamento em y[2]:
`print("sw $s0, ", -k, "($s1)")` # onde $k = (p - 1 + indice) * 4$
 $= ((2 - 1) + 2) * 4 = 12$

Implementação de escopos em bloco

- Na linguagem G-V2 e na linguagem C pode ocorrer aninhamento de blocos.
- Uma variável x declarada em um bloco pode ser acessada em um bloco mais interno, a menos que outra variável com mesmo nome seja declarada no bloco mais interno.
- O aninhamento de blocos segue também o comportamento de uma pilha: as variáveis do bloco mais externo devem aparecer mais cedo na pilha e as variáveis do bloco mais interno devem aparecer no topo da pilha. Quando termina o bloco mais interno, suas variáveis são desempilhadas.

Acesso a variáveis de blocos aninhados

Exemplo

```
int x,y; //variáveis globais:  $x_{global}, y_{global}$ 
int f(){
    int a,b; //  $a_{f_1}, b_{f_1}$ 
    ...
    { int a,c; //  $a_{f_2}, c_{f_2}$ 
      ... a=2; }
}
programa{
    int a,b; //  $a_{prog}, b_{prog}$ 
    ...
    f();
    ...
}
```

Acesso a variáveis de blocos aninhados

Imagem da memória

- Imagem da memória quando o bloco mais interno de f está ativo:

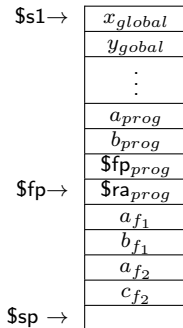


Fig. 1

Sugestão

- As variáveis são alocadas na pilha na sequência em que aparecem nas declarações. Ver figura anterior.
- As variáveis declaradas no bloco mais externo na função têm escopo igual a 1 (0 é o escopo das variáveis globais).
- A cada bloco mais interno o escopo é aumentado de um.

Sugestão

- Durante a geração de código para uma função:
 - a variável *escopo* é iniciada com 1 no bloco principal da função. Essa variável é aumentada com 1 sempre que novo bloco é criado e diminuída de 1 quando um bloco termina.
 - Para cada bloco, se há uma declaração de variáveis no bloco, criamos uma nova tabela no topo da pilha de tabela de símbolos.
 - Para cada variável encontrada na lista de variáveis, inserimos seu nome, escopo e posição na tabela de símbolos correspondente. Porém, essa posição é a posição da variável em relação à primeira variável da função e não em relação à primeira variável da lista onde ela foi declarada.

Sugestão - continuação

- por exemplo, considere a pilha de tabela de símbolos existente quando a geração de código atingir a atribuição: $a=2$; no último exemplo:

a	$\langle escopo = 2, pos = 3, \dots \rangle$
c	$\langle escopo = 2, pos = 4, \dots \rangle$
a	$\langle escopo = 1, pos = 1, \dots \rangle$
b	$\langle escopo = 1, pos = 2, \dots \rangle$
x	$\langle escopo = 0, pos = 1, \dots \rangle$
y	$\langle escopo = 0, pos = 2, \dots \rangle$

- Ou seja, a posição da variável a no bloco mais interno de $f()$ é 3.

Sugestão - continuação

- Ao final de cada declaração dentro de um bloco, emitimos código para deslocar o \$sp o número de bytes correspondente ao número de variáveis da declaração. Por exemplo, ao final da declaração `int a, b;`: `print (addiu $sp, $p, -8);` para reservar espaço para as duas variáveis da declaração
- Ao final da declaração do bloco mais interno, a configuração da memória é aquela mostrada na Fig. 1.
- O acesso à uma variável local à uma função é obtido com referência ao *frame pointer*. Por exemplo para realizar a atribuição `a=2`, consultamos *a* na tabela de símbolos correspondente ao escopo dois e descobrimos que sua posição é 3. Então o código correspondente seria: `print("li $s0, 2");`
`desloc=4*3;print("sw $s0, (-desloc)$fp).`

Acesso a variáveis em blocos aninhados - Exemplo

Exemplo de código

```
int x,y; //variáveis globais
int f(){
    int a,b; //  $a_f, b_f$ 
    ...
    { int a,c; //  $a_{f_1}, c_f$ 
    ... a=2; }
}
programa{
    int a,b;
    ...
    f();
    ...
}
```

Memória

\$s1 →	x_{global}
	y_{global}
	$\vdots$
	a_{prog}
	b_{prog}
	$\$fp_{prog}$
\$fp →	$\$ra_{prog}$
	a_{f_1}
	b_{f_1}
	a_{f_2}
	c_{f_2}
\$sp →	

Acesso a a e c

Ao gerar código para $a=2$; no bloco mais interno de f :

- Consulta-se a tabela de símbolos com o nome a ;
- A consulta retorna informações sobre a_{f_2} :
 $escopo(a_{f_2}) = 2$ e
 $pos(a_{f_2}) = 3$.
- O acesso a a (a_{f_2} na figura) é dado por: $\$fp - (pos(a_{f_2}) * 4) = \$fp - 12$.
- De modo semelhante, o acesso a c (c_f na figura) é dado por $\$fp - (pos(c_f)) * 4 = \$fp - 16$

System Calls

- Os simuladores SPIM e Mars possuem chamadas ao sistema para escrita de números e strings.
- Cada chamada possui como código um valor inteiro a ser colocado no registrador \$v0 antes da chamada.
- A tabela abaixo mostra as principais tarefas de entrada e saída realizada por esses simuladores:

Código	Argumentos de entrada	Resultado
1	\$a0 = valor inteiro	Imprime valor inteiro
4	\$a0=string de caracteres	Imprime cadeia
5		Lê valor inteiro em \$v0

- O valor a ser operado fica em \$a0.

Exemplo

- Considere os seguinte comandos:
 escreva("A resposta e: "); escreva (5);
- O código correspondente em MIPS:

```
.data
str:
    .asciiz "A resposta e: "
    .text
li $v0, 4 # código de chamada para impressao de string
la, $a0, str # carrega em $a0 o endereço da string
syscall #executa a chamda ao sistema
#
li $v0, 1 # código para imprimir um valor inteiro
li $a0, 5 # carrega em $a0 o valor a ser impresso
syscall
```

Exemplo

- Considere o seguinte comando:

```
leia(x);
```

- O código correspondente em MIPS:

```
li $v0, 5 # código de chamada para leitura de um inteiro
syscall #executa a chamada ao sistema
# armazene o valor em $v0 no endereço da pilha correspondente a x
```